Air University

Data Structures & Algorithms (Lab)

Year 2025

Mr. Ali Raza

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: October 17, 2025

Assignment # 01

Task A:

Code

Row (1-32)

```
#include <iostream>
using namespace std;

class Node // Node class represents a single element in the singly linked list
{
public:
    int data;
    Node *next;
};

class SinglyList // SinglyList class implements a singly linked list with various operations
{
private:
    Node *head = nullptr;
    int counter = 0;

public:
    Node *createNode() // Function to create a new node with user input
    {
        int val;
        cout << "Enter a Number: ";
        cin >> val;

        Node *newNode = new Node(); // Allocate memory for new node

        newNode->data = val;
        newNode->next = nullptr;
        counter++;

        return newNode;
    }

```

Row (33-49)

```
    void add_At_Beginning() // Function to add a new node at the beginning of the list
    {
        Node *newNode = createNode();

        if (head == nullptr) // If list is empty
        {
            head = newNode; // Set head to new node
        }
        else
        {
            newNode->next = head; // Point new node to current head
            head = newNode;       // Update head to new node
        }
        cout << "New Node Added at Beginning. " << endl;
        system("pause");
    }

```

Row (50-72)

```
50      void add_At_End() // Function to add a new node at the end of the list
51      {
52          Node *newNode = createNode();
53
54          if (head == nullptr) // If list is empty
55          {
56              head = newNode; // Set head to new node
57          }
58          else
59          {
60              Node *current = head; // Start from head
61
62              while (current->next != nullptr) // Traverse to the last node
63              {
64                  current = current->next;
65              }
66              current->next = newNode; // Link new node at the end
67          }
68
69          cout << "New Node Added at End." << endl;
70          system("pause");
71      }
72
```

Row (73-101)

```
73      void add_At_Position() // Function to add a new node at a specific position in the list
74      {
75          int pos;
76          cout << "Enter Desired Location: ";
77          cin >> pos;
78
79          if (head == nullptr) // If list is empty
80          {
81              cout << "Playlist is Empty. " << endl;
82              system("pause");
83              return;
84          }
85          if (pos < 1 || pos > counter + 1) // Check for invalid position
86          {
87              cout << "Invalid Position. " << endl;
88              system("pause");
89              return;
90          }
91          if (pos == 1) // If position is 1, add at beginning
92          {
93              add_At_Beginning();
94              return;
95          }
96          if (pos == counter + 1) // If position is at end, add at end
97          {
98              add_At_End();
99              return;
100         }
101
```

Row(102-130)

```
102         Node *current = head; // Start from head
103         int index = 1;
104 
105         while (current->next != nullptr && index++ < pos) // Traverse to position-1
106         {
107             current = current->next;
108         }
109         Node *temp = createNode();
110 
111         temp->next = current->next; // Link new node to next
112         current->next = temp;       // Link current to new node
113 
114         cout << "Node Added at Desired Location. " << endl;
115         system("pause");
116     }
117 
118     void delete_By_Position() // Function to delete a node at a specific position
119     {
120         int pos;
121         cout << "Enter Desired Position: ";
122         cin >> pos;
123 
124         if (pos <= 0) // Invalid position check
125         {
126             cout << "Invalid Position (must be greater than 0)." << endl;
127             system("pause");
128             return;
129         }
130 
```

Row (131-159)

```
131         if (head == nullptr) // If list is empty
132         {
133             cout << "List is Empty." << endl;
134             system("pause");
135             return;
136         }
137 
138         if (pos == 1) // If deleting head
139         {
140             Node *temp = head;
141             head = head->next;
142             delete temp;
143             counter--;
144             cout << "Node at Position " << pos << " Deleted Successfully." << endl;
145             system("pause");
146             return;
147         }
148 
149         Node *current = head;
150         Node *previous = nullptr;
151         int index = 1;
152 
153         while (current != nullptr && index < pos) // Traverse to the position
154         {
155             previous = current;
156             current = current->next;
157             index++;
158         }
159 
```

Row (160-187)

```
160         if (current == nullptr) // If position not found
161         {
162             cout << "Invalid Position." << endl;
163             system("pause");
164             return;
165         }
166
167         previous->next = current->next; // Bypass the node
168         delete current;
169         counter--;
170
171         cout << "Node at Position " << pos << " Deleted Successfully." << endl;
172         system("pause");
173     }
174
175     void delete_Value() // Function to delete a node by value
176     {
177         int val;
178         cout << "Enter Value for Deletion: ";
179         cin >> val;
180
181         if (head == nullptr) // If list is empty
182         {
183             cout << "List is Empty." << endl;
184             system("pause");
185             return;
186         }
187
```

Row (188-217)

```
188         Node *current = head;
189         Node *previous = nullptr;
190
191         while (current != nullptr && current->data != val) // Search for the value
192         {
193             previous = current;
194             current = current->next;
195         }
196
197         if (current == nullptr) // If value not found
198         {
199             cout << "value " << val << " Not Found in the List." << endl;
200             system("pause");
201             return;
202         }
203
204         if (previous == nullptr) // If deleting head
205         {
206             head = head->next;
207         }
208         else
209         {
210             previous->next = current->next;
211         }
212         delete current;
213
214         cout << "Value " << val << " Deleted Successfully." << endl;
215         system("pause");
216     }
217
```

Row (218-246)

```
218     void count_Nodes() // Function to count and display the number of nodes
219     {
220         cout << "\nNumber of Nodes: " << counter << endl;
221         system("pause");
222     }
223
224     void display_list() // Function to display the entire list
225     {
226         if (head == nullptr) // If list is empty
227         {
228             cout << "\nList is Empty." << endl;
229             system("pause");
230             return;
231         }
232         else
233         {
234             cout << "\n\tSingly List" << endl;
235             Node *temp = head;
236
237             while (temp != nullptr) // Traverse and print each node
238             {
239                 cout << temp->data << endl;
240                 temp = temp->next;
241             }
242             cout << "End of List. " << endl;
243         }
244         system("pause");
245     }
246
```

Row (247-276)

```
247 void reverse_Iterative() // Function to reverse the list iteratively
248 {
249     Node *current = head;
250     Node *previous = nullptr;
251     Node *nextNode = nullptr;
252
253     while (current != nullptr) // Iterate through the list
254     {
255         nextNode = current->next; // Store next node
256         current->next = previous; // Reverse the link
257         previous = current;       // Move previous forward
258         current = nextNode;       // Move current forward
259     }
260     head = previous; // Update head to new first node
261
262     cout << "\nList successfully reversed Iterativley. " << endl;
263     display_list();
264 }
265
266 Node *reverse_Helper(Node *current, Node *previous) // Helper function for recursive reversal
267 {
268     if (current == nullptr) // Base case: end of list
269     {
270         return previous;
271     }
272     Node *nextNode = current->next;           // Store next
273     current->next = previous;                 // Reverse link
274     return reverse_Helper(nextNode, current); // Recurse
275 }
276
```

Row (277-310)

```
277     void reverse_Recursively() // Function to reverse the list recursively
278     {
279         head = reverse_Helper(head, nullptr); // Call helper with head and null
280
281         cout << "\nList successfully reversed recursively." << endl;
282         display_list();
283     }
284 };
285
286 int main()
287 {
288     SinglyList list1; // Create an instance of SinglyList
289
290     while (1) // Infinite loop for menu
291     {
292         system("cls"); // Clear screen
293
294         cout << "***    Singly list    ***\n"
295              << "1. Add Node at Beginning.\n"
296              << "2. Add Node at End.\n"
297              << "3. Add Node at Specific Location. \n"
298              << "4. Delete Node from Specific Location.\n"
299              << "5. Delete Value from Index.\n"
300              << "6. Count Nodes.\n"
301              << "7. Display list.\n"
302              << "8. Display Iterative Reverse.\n"
303              << "9. Display Recursive Reverse.\n"
304              << "0. Exit\n"
305              << "\nEnter your choice: ";
306
307         int choice;
308         cin >> choice;
309         cin.ignore(); // Ignore newline
310
```

Row (311-350)

```
311             switch (choice) // Handle user choice
312             {
313             case 1:
314                 list1.add_At_Beginning();
315                 break;
316             case 2:
317                 list1.add_At_End();
318                 break;
319             case 3:
320                 list1.add_At_Position();
321                 break;
322             case 4:
323                 list1.delete_By_Position();
324                 break;
325             case 5:
326                 list1.delete_Value();
327                 break;
328             case 6:
329                 list1.count_Nodes();
330                 break;
331             case 7:
332                 list1.display_list();
333                 break;
334             case 8:
335                 list1.reverse_Iterative();
336                 break;
337             case 9:
338                 list1.reverse_Recursively();
339                 break;
340             case 0:
341                 exit(0); // Exit program
342             default:
343                 cout << "Invalid choice" << endl;
344                 system("pause");
345                 break;
346             }
347         }
348         cout << endl;
349         return 0;
350     }
```

Output

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 1
Enter a Number: 2
New Node Added at Beginning.
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 2
Enter a Number: 4
New Node Added at End.
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 3
Enter Desired Location: 3
Enter a Number: 6
New Node Added at End.
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 4
Enter Desired Position: 1
Node at Position 1 Deleted Successfully.
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 5
Enter Value for Deletion: 6
Value 6 Deleted Successfully.
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 6

Number of Nodes: 2
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 7

        Singly List
2
4
6
End of List.
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 8

List successfully reversed Iterativley.

        Singly List
6
4
2
End of List.
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 9

List successfully reversed recursively.

        Singly List
2
4
6
End of List.
Press any key to continue . . .
```

```
***    Singly list    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
0. Exit

Enter your choice: 0
PS D:\University\DSA\Lab\Assignment 1>
```

Task B:

Code

Row (1-33)

```
#include <iostream>
using namespace std;

class Node // Represents a single element in the doubly linked list
{
public:
    int data;
    Node *next;
    Node *prev;
};

class DoublyList // Implements a doubly linked list with various operations
{
private:
    Node *head = nullptr;
    Node *tail = nullptr;
    int counter = 0;

public:
    Node *createNode() // Function to create a new node with user input
    {
        int val;
        cout << "Enter a Number: ";
        cin >> val;

        Node *newNode = new Node(); // Allocate memory for new node
        newNode->data = val;
        newNode->next = nullptr;
        newNode->prev = nullptr;
        counter++;
        return newNode;
    }

```

Row (34-53)

```
    void add_At_Beginning() // Function to add a new node at the beginning of the list
    {
        Node *newNode = createNode();

        if (head == nullptr) // If list is empty
        {
            head = newNode; // Set head to new node
            tail = newNode; // Also set tail to new node
        }
        else
        {
            newNode->next = head;    // Point new node to current head
            head->prev = newNode;    // Set prev of current head to new node
            newNode->prev = nullptr; // New node's prev is nullptr
            head = newNode;          // Update head to new node
        }
        cout << "New Node Added at Beginning." << endl;
        system("pause");
    }

```

Row (54-73)

```
54      void add_At_End() // Function to add a new node at the end of the list
55      {
56          Node *newNode = createNode();
57
58          if (tail == nullptr) // If list is empty
59          {
60              head = newNode; // Set head to new node
61              tail = newNode; // Set tail to new node
62          }
63          else
64          {
65              tail->next = newNode;    // Link current tail to new node
66              newNode->prev = tail;    // Set prev of new node to current tail
67              tail = newNode;          // Update tail to new node
68              newNode->next = nullptr; // New node's next is nullptr
69          }
70          cout << "New Node Added at End." << endl;
71          system("pause");
72      }
73
```

Row (74-102)

```
74      void add_At_Position() // Function to add a new node at a specific position in the list
75      {
76          int pos;
77          cout << "Enter Desired Location: ";
78          cin >> pos;
79
80          if (head == nullptr) // If list is empty
81          {
82              cout << "List is Empty." << endl;
83              system("pause");
84              return;
85          }
86          if (pos < 1 || pos > counter + 1) // Check for invalid position
87          {
88              cout << "Invalid Position." << endl;
89              system("pause");
90              return;
91          }
92          if (pos == 1) // If position is 1, add at beginning
93          {
94              add_At_Beginning();
95              return;
96          }
97          if (pos == counter + 1) // If position is at end, add at end
98          {
99              add_At_End();
100             return;
101         }
102
```

Row (103-124)

```
103         Node *current = head; // Start from head
104         int index = 1;
105         while (current->next != nullptr && index++ < pos - 1) // Traverse to position-1
106         {
107             current = current->next;
108         }
109         Node *newNode = createNode();
110         newNode->next = current->next; // Link new node to next
111         if (current->next != nullptr)  // If there's a node after current
112         {
113             current->next->prev = newNode; // Set prev of the next node
114         }
115         current->next = newNode;       // Link current to new node
116         newNode->prev = current;       // Set prev of new node
117         if (newNode->next == nullptr) // If new node is now the last
118         {
119             tail = newNode; // Update tail
120         }
121         cout << "Node Added at Desired Location." << endl;
122         system("pause");
123     }
124
```

Row (125-163)

```
125     void delete_By_Position() // Function to delete a node at a specific position
126     {
127         int pos;
128         cout << "Enter Desired Position: ";
129         cin >> pos;
130
131         if (pos <= 0) // Invalid position check
132         {
133             cout << "Invalid Position (must be greater than 0)." << endl;
134             system("pause");
135             return;
136         }
137
138         if (head == nullptr) // If list is empty
139         {
140             cout << "List is Empty." << endl;
141             system("pause");
142             return;
143         }
144
145         if (pos == 1) // If deleting head
146         {
147             Node *temp = head;
148             head = head->next;
149             if (head != nullptr)
150             {
151                 head->prev = nullptr; // Set prev of new head to nullptr
152             }
153             else
154             {
155                 tail = nullptr; // List is now empty, so tail is nullptr
156             }
157             delete temp;
158             counter--;
159             cout << "Node at Position " << pos << " Deleted Successfully." << endl;
160             system("pause");
161             return;
162         }
163
```

Row (164-193)

```
164         Node *current = head;
165         int index = 1;
166         while (current != nullptr && index < pos) // Traverse to the position
167         {
168             current = current->next;
169             index++;
170         }
171 
172         if (current == nullptr) // If position not found
173         {
174             cout << "Invalid Position." << endl;
175             system("pause");
176             return;
177         }
178 
179         if (current->next != nullptr)
180         {
181             current->next->prev = current->prev; // Update prev of next node
182         }
183         else
184         {
185             tail = current->prev; // Update tail if deleting the last node
186         }
187         current->prev->next = current->next; // Bypass the node
188         delete current;
189         counter--;
190         cout << "Node at Position " << pos << " Deleted Successfully." << endl;
191         system("pause");
192     }
193 
```

Row (194-220)

```
194     void delete_Value() // Function to delete a node by value
195     {
196         int val;
197         cout << "Enter Value for Deletion: ";
198         cin >> val;
199 
200         if (head == nullptr) // If list is empty
201         {
202             cout << "List is Empty." << endl;
203             system("pause");
204             return;
205         }
206 
207         Node *current = head;
208 
209         while (current != nullptr && current->data != val) // Search for the value
210         {
211             current = current->next;
212         }
213 
214         if (current == nullptr) // If value not found
215         {
216             cout << "Value " << val << " Not Found in the List." << endl;
217             system("pause");
218             return;
219         }
220 
```

Row (221-252)

```
221         if (current->prev != nullptr) // If not the head
222         {
223             current->prev->next = current->next; // Update next of previous node
224         }
225         else // If it is the head
226         {
227             head = current->next; // Update head
228             if (head != nullptr)
229             {
230                 head->prev = nullptr; // Set prev of new head to nullptr
231             }
232             else
233             {
234                 tail = nullptr; // List is now empty
235             }
236         }
237 
238         if (current->next != nullptr) // If not the last node
239         {
240             current->next->prev = current->prev; // Update prev of next node
241         }
242         else
243         {
244             tail = current->prev; // Update tail if deleting the last node
245         }
246 
247         delete current;
248         counter--;
249         cout << "Value " << val << " Deleted Successfully." << endl;
250         system("pause");
251     }
252 
```

Row (253-280)

```
253     void insert_before_value() // Insert a new node before the first node with a specified value
254     {
255         int x, new_val;
256         cout << "Enter the value to insert before: ";
257         cin >> x;
258         cout << "Enter the new value: ";
259         cin >> new_val;
260 
261         if (head == nullptr) // If list is empty
262         {
263             cout << "List is Empty. Cannot insert." << endl;
264             system("pause");
265             return;
266         }
267 
268         Node *current = head;
269         while (current != nullptr && current->data != x) // Search for the value x
270         {
271             current = current->next;
272         }
273 
274         if (current == nullptr) // If value x is not found
275         {
276             cout << "Value " << x << " Not Found in the List." << endl;
277             system("pause");
278             return;
279         }
280 
```

Row (281-303)

```
281         Node *newNode = new Node(); // Create a new node
282         newNode->data = new_val;
283         newNode->next = current;       // Link new node to current
284         newNode->prev = current->prev; // Link new node's prev to current's prev
285
286         if (current->prev != nullptr) // If current is not the head
287         {
288             current->prev->next = newNode; // Update previous node's next
289         }
290         else // If current is the head
291         {
292             head = newNode; // Update head to new node
293         }
294         current->prev = newNode;      // Update current's prev to new node
295         if (newNode->next == nullptr) // If new node is now the last
296         {
297             tail = newNode; // Update tail
298         }
299         counter++; // Increment counter
300         cout << "New Node Inserted Before Value " << x << "." << endl;
301         system("pause");
302     }
303
```

Row (304-336)

```
304 void delete_before_value() // Function to delete the node before the first node with a specified value
305 {
306     int x;
307     cout << "Enter the value before which to delete: ";
308     cin >> x;
309
310     if (head == nullptr) // If list is empty
311     {
312         cout << "List is Empty. Cannot delete." << endl;
313         system("pause");
314         return;
315     }
316
317     Node *current = head;
318     while (current != nullptr && current->data != x) // Search for the value x
319     {
320         current = current->next;
321     }
322
323     if (current == nullptr) // If value x is not found
324     {
325         cout << "Value " << x << " Not Found in the List." << endl;
326         system("pause");
327         return;
328     }
329
330     if (current->prev == nullptr) // If the node with x is the head
331     {
332         cout << "No node before the first occurrence of " << x << "." << endl;
333         system("pause");
334         return;
335     }
336
```

Row (337-358)

```
337         Node *nodeToDelete = current->prev; // Node to delete is the one before current
338
339         if (nodeToDelete->prev != nullptr) // If the node to delete is not the head
340         {
341             nodeToDelete->prev->next = nodeToDelete->next; // Update previous node's next
342         }
343         else // If the node to delete is the head
344         {
345             head = nodeToDelete->next; // Update head
346             head->prev = nullptr;      // Set new head's prev to nullptr
347         }
348         nodeToDelete->next->prev = nodeToDelete->prev; // Update next node's prev
349         if (nodeToDelete == tail)                      // If the node to delete was the tail
350         {
351             tail = nodeToDelete->prev; // Update tail
352         }
353         delete nodeToDelete; // Delete the node
354         counter--;           // Decrement counter
355         cout << "Node Before Value " << x << " Deleted Successfully." << endl;
356         system("pause");
357     }
358
```

Row (359-384)

```
359     void display_reverse() // Function to display the list in reverse order
360     {
361         if (head == nullptr) // If list is empty
362         {
363             cout << "\nList is Empty." << endl;
364             system("pause");
365             return;
366         }
367
368         Node *current = tail; // Start from tail for reverse display
369         cout << "\n\tDoubly List in Reverse:" << endl;
370         while (current != nullptr) // Traverse backwards
371         {
372             cout << current->data << endl;
373             current = current->prev;
374         }
375         cout << "End of Reverse List." << endl;
376         system("pause");
377     }
378
379     void count_Nodes() // Function to count and display the number of nodes
380     {
381         cout << "\nNumber of Nodes: " << counter << endl;
382         system("pause");
383     }
384
```

Row (385-406)

```
385     void display_list() // Function to display the entire list
386     {
387         if (head == nullptr) // If list is empty
388         {
389             cout << "\nList is Empty." << endl;
390             system("pause");
391             return;
392         }
393         else
394         {
395             cout << "\n\tDoubly List" << endl;
396             Node *temp = head;
397             while (temp != nullptr) // Traverse and print each node
398             {
399                 cout << temp->data << endl;
400                 temp = temp->next;
401             }
402             cout << "End of List." << endl;
403         }
404         system("pause");
405     }
406
```

Row (407-437)

```
407     void reverse_Iterative() // Function to reverse the list iteratively
408     {
409         Node *current = head;
410         Node *temp = nullptr;
411
412         while (current != nullptr)
413         {
414             temp = current->prev;          // Save prev
415             current->prev = current->next; // Swap prev and next
416             current->next = temp;          // Complete swap
417             current = current->prev;       // Move to the original next (now prev)
418         }
419
420         if (temp != nullptr)
421         {
422             head = temp->prev;            // Update head
423             tail = head;                  // Reset tail to original head (now last)
424             while (tail->next != nullptr) // Traverse to new tail
425             {
426                 tail = tail->next;
427             }
428         }
429         else
430         {
431             tail = nullptr; // List is empty
432         }
433
434         cout << "\nList successfully reversed Iteratively." << endl;
435         display_list();
436     }
437
```

Row (438-464)

```
438     Node *reverse_Helper(Node *current) // Helper function for recursive reversal
439     {
440         if (current == nullptr) // Base case
441             return nullptr;
442 
443         // Swap next and prev
444         Node *temp = current->next;    // Store original next pointer
445         current->next = current->prev; // Swap next to previous node
446         current->prev = temp;          // Swap prev to original next
447 
448         if (current->prev == nullptr) // If this is the new head
449         {
450             tail = current; // Set tail to current (now last node)
451             return current; // Return the new head
452         }
453 
454         return reverse_Helper(current->prev); // Recurse with the original next
455     }
456 
457     void reverse_Recursively() // Function to reverse the list recursively
458     {
459         head = reverse_Helper(head); // Get the new head
460         cout << "\nList successfully reversed recursively." << endl;
461         display_list();
462     }
463 };
464 
```

Row (465-492)

```
465 int main()
466 {
467     DoublyList list1; // Create an instance of DoublyList
468 
469     while (1) // Infinite loop for menu
470     {
471         system("cls"); // Clear screen
472 
473         cout << "***    Doubly Linked List    ***\n"
474              << "1. Add Node at Beginning.\n"
475              << "2. Add Node at End.\n"
476              << "3. Add Node at Specific Location.\n"
477              << "4. Delete Node from Specific Location.\n"
478              << "5. Delete Value from Index.\n"
479              << "6. Count Nodes.\n"
480              << "7. Display list.\n"
481              << "8. Display Iterative Reverse.\n"
482              << "9. Display Recursive Reverse.\n"
483              << "10. Display List in Reverse.\n"
484              << "11. Insert Before Value.\n"
485              << "12. Delete Before Value.\n"
486              << "0. Exit\n"
487              << "\nEnter your choice: ";
488 
489         int choice;
490         cin >> choice;
491         cin.ignore(); // Ignore newline
492 
```

Row (493-540)

```
493         switch (choice)
494         {
495         case 1:
496             list1.add_At_Beginning();
497             break;
498         case 2:
499             list1.add_At_End();
500             break;
501         case 3:
502             list1.add_At_Position();
503             break;
504         case 4:
505             list1.delete_By_Position();
506             break;
507         case 5:
508             list1.delete_Value();
509             break;
510         case 6:
511             list1.count_Nodes();
512             break;
513         case 7:
514             list1.display_list();
515             break;
516         case 8:
517             list1.reverse_Iterative();
518             break;
519         case 9:
520             list1.reverse_Recursively();
521             break;
522         case 10:
523             list1.display_reverse();
524             break;
525         case 11:
526             list1.insert_before_value();
527             break;
528         case 12:
529             list1.delete_before_value();
530             break;
531         case 0:
532             exit(0);
533         default:
534             cout << "Invalid choice" << endl;
535             system("pause");
536             break;
537         }
538     }
539     return 0;
540 }
```

Output

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 1
Enter a Number: 6
New Node Added at Beginning.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 2
Enter a Number: 6
New Node Added at End.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 3
Enter Desired Location: 3
Enter a Number: 7
New Node Added at End.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 4
Enter Desired Position: 1
Node at Position 1 Deleted Successfully.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 5
Enter Value for Deletion: 7
Value 7 Deleted Successfully.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 6

Number of Nodes: 1
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 7

        Doubly List
5
6
6
End of List.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 8

List successfully reversed Iteratively.

        Doubly List
5
6
7
End of List.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 9

List successfully reversed recursively.

        Doubly List
7
6
5
End of List.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 10

        Doubly List in Reverse:
5
6
7
End of Reverse List.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 11
Enter the value to insert before: 5
Enter the new value: 4
New Node Inserted Before Value 5.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 12
Enter the value before which to delete: 4
Node Before Value 4 Deleted Successfully.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 7

        Doubly List
7
4
5
End of List.
Press any key to continue . . .
```

```
***    Doubly Linked List    ***
1. Add Node at Beginning.
2. Add Node at End.
3. Add Node at Specific Location.
4. Delete Node from Specific Location.
5. Delete Value from Index.
6. Count Nodes.
7. Display list.
8. Display Iterative Reverse.
9. Display Recursive Reverse.
10. Display List in Reverse.
11. Insert Before Value.
12. Delete Before Value.
0. Exit

Enter your choice: 0
PS D:\University\DSA\Lab\Assignment 1>
```

Task C:

Code

Row (1-32)

```
1   #include <iostream>
2   using namespace std;
3
4   class Node // Node class for doubly linked list elements
5   {
6   public:
7       int data;
8       Node *next;
9       Node *prev;
10  };
11
12  class CircularList // Circular doubly linked list class
13  {
14  private:
15      Node *head = nullptr;
16      Node *tail = nullptr;
17
18  public:
19      Node *createNode() // Creates a new node with user input
20      {
21          int val;
22          cout << "Enter a Number: ";
23          cin >> val;
24
25          Node *newNode = new Node();
26          newNode->data = val;
27          newNode->next = nullptr;
28          newNode->prev = nullptr;
29
30          return newNode;
31      }
32
```

Row (33-55)

```
33      void add_At_End() // Adds a new node at the end of the list
34      {
35          Node *newNode = createNode();
36
37          if (tail == nullptr) // If list is empty
38          {
39              head = newNode;       // Set head to new node
40              tail = newNode;       // Set tail to new node
41              newNode->next = head; // Point next to itself
42              newNode->prev = tail; // Point prev to itself
43          }
44          else // List not empty
45          {
46              tail->next = newNode; // Link new node after tail
47              newNode->prev = tail; // Set new node's prev to tail
48              tail = newNode;       // Update tail to new node
49              tail->next = head;    // Maintain circular link
50              head->prev = tail;    // Update head's prev
51          }
52          cout << "New Node Added at End." << endl;
53          system("pause");
54      }
55
```

Row (56-84)

```
56      void delete_value() // Deletes a node with the specified value
57      {
58          if (head == nullptr) // Check if list is empty
59          {
60              cout << "List is empty. Cannot delete." << endl;
61              system("pause");
62              return;
63          }
64          int value;
65          cout << "Enter the value to delete: ";
66          cin >> value;
67          Node *current = head;
68          do
69          {
70              if (current->data == value)
71              {
72                  if (head == tail) // Only one node
73                  {
74                      head = nullptr;
75                      tail = nullptr;
76                  }
77                  else if (current == head) // Deleting head
78                  {
79                      if (head == nullptr) // Redundant check
80                      {
81                          cout << "List is empty. Cannot delete." << endl;
82                          return;
83                      }
84
```

Row (85-111)

```
85                  Node *temp = head;
86
87                  if (head == tail)
88                  {
89                      head = nullptr;
90                      tail = nullptr;
91                  }
92                  else
93                  {
94                      head = head->next; // Move head to next node
95                      tail->next = head; // Update tail's next to new head
96                      head->prev = tail; // Update new head's prev to tail
97                  }
98
99                  delete temp;
100                 cout << "Node deleted at first." << endl;
101                 system("pause");
102                 return;
103             }
104             else if (current == tail) // Deleting tail
105             {
106                 if (tail == nullptr) // Redundant check
107                 {
108                     cout << "List is empty. Cannot delete." << endl;
109                     return;
110                 }
111
```

Row (112-147)

```
112                 Node *temp = tail;
113
114                 if (head == tail)
115                 {
116                     head = nullptr;
117                     tail = nullptr;
118                 }
119                 else
120                 {
121                     tail = tail->prev; // Move tail to previous node
122                     tail->next = head; // Update new tail's next to head
123                     head->prev = tail; // Update head's prev to new tail
124                 }
125
126                 delete temp;
127                 cout << "Node deleted at last." << endl;
128                 system("pause");
129                 return;
130             }
131             else // Deleting a middle node
132             {
133                 current->prev->next = current->next; // Bypass current node in next direction
134                 current->next->prev = current->prev; // Bypass current node in prev direction
135             }
136             delete current;
137             cout << "Node with value " << value << " deleted." << endl;
138             system("pause");
139             return;
140         }
141         current = current->next; // Move to next node
142     } while (current != head); // Loop until back to head
143
144     cout << "Value " << value << " not found in the list." << endl;
145     system("pause");
146 }
147
```

Row (148-167)

```
148     void displayList() // Displays the list
149     {
150         if (head == nullptr) // Check if list is empty
151         {
152             cout << "List is empty." << endl;
153             return;
154         }
155 
156         Node *current = head;
157         cout << "Circular Linked List: ";
158         do
159         {
160             cout << current->data << " ";
161             current = current->next;
162         } while (current != head); // Traverse until back to head
163         cout << endl;
164         system("pause");
165     }
166 };
167 
```

Row (168-200)

```
168 int main()
169 {
170     CircularList list;
171     int choice;
172 
173     while (true) // Infinite loop for menu
174     {
175         cout << "\n1. Add at End\n"
176              << "2. Delete Value\n"
177              << "3. Display List\n"
178              << "4. Exit\n";
179         cout << "Enter your choice: ";
180         cin >> choice;
181 
182         switch (choice)
183         {
184         case 1:
185             list.add_At_End();
186             break;
187         case 2:
188             list.delete_value();
189             break;
190         case 3:
191             list.displayList();
192             break;
193         case 4:
194             return 0;
195         default:
196             cout << "Invalid choice." << endl;
197         }
198     }
199     return 0;
200 }
```

Output

```
1. Add at End
2. Delete Value
3. Display List
4. Exit
Enter your choice: 1
Enter a Number: 2
New Node Added at End.
Press any key to continue . . .

1. Add at End
2. Delete Value
3. Display List
4. Exit
Enter your choice: 1
Enter a Number: 3
New Node Added at End.
Press any key to continue . . .
```

```
1. Add at End
2. Delete Value
3. Display List
4. Exit
Enter your choice: 1
Enter a Number: 4
New Node Added at End.
Press any key to continue . . .

1. Add at End
2. Delete Value
3. Display List
4. Exit
Enter your choice: 2
Enter the value to delete: 3
Node with value 3 deleted.
Press any key to continue . . .
```

```
1. Add at End
2. Delete Value
3. Display List
4. Exit
Enter your choice: 3
Circular Linked List: 2 4
Press any key to continue . . .

1. Add at End
2. Delete Value
3. Display List
4. Exit
Enter your choice: 4
PS D:\University\DSA\Lab\Assignment 1>
```

THE END